```python
pages = list(map(int, input("Enter page reference string: ").split()))

frames = int(input("Enter number of frames: "))

memory = []
page_faults = 0

print("\nPage\tFrames")

for page in pages:

    if page not in memory:

        page_faults += 1

        if len(memory) < frames:

            memory.append(page)

        else:

            memory.pop(0)
            memory.append(page)

    print(page, "\t", memory)

print("\nTotal Page Faults:", page_faults)
```

Output

```
= RESTART: C:/Users/ATHIN/AppData/Local/Programs/Python/Python313
sssssssssssssssssssssssssssssssssssssssssssssssssssss.py
Enter page reference string: 4
Enter number of frames: 4

Page    Frames
34       [4]

Total Page Faults: 1
```

```
pages = list(map(int, input("Enter page reference string: ").split()))

frames = int(input("Enter number of frames: "))

memory = []
page_faults = 0

print("\nPage\tFrames")

for page in pages:

    if page in memory:

        memory.remove(page)
        memory.append(page)

    else:

        page_faults += 1

        if len(memory) < frames:

            memory.append(page)

        else:

            memory.pop(0)
            memory.append(page)

    print(page, "\t", memory)

print("\nTotal Page Faults:", page_faults)
```

Output

```
= RESTART: C:/Users/ATHIN/AppData/Local/Pro
sssssssssssssssssssssssssssssssssssssssssss
Enter page reference string: 4
Enter number of frames: 34

Page    Frames
4         [4]

Total Page Faults: 1
```

```
print("\nPage\tFrames")

for i in range(len(pages)):

    page = pages[i]

    if page not in memory:

        page_faults += 1

        if len(memory) < frames:

            memory.append(page)

        else:

            future = pages[i + 1:]

            index = -1
            farthest = -1

            for j in range(len(memory)):

                if memory[j] not in future:

                    index = j
                    break

                position = future.index(memory[j])

                if position > farthest:

                    farthest = position
                    index = j

            memory[index] = page

    print(page, "\t", memory)

print("\nTotal Page Faults:", page_faults)
```

OUTPUT

```
= RESTART: C:/Users/ATHIN/AppData/L
sssssssssssssssssssssssssssssssssss
Enter page reference string: 4
Enter number of frames: 34

Page    Frames
4         [4]

Total Page Faults: 1
```

```
blocks = list(map(int, input("Enter available disk blocks: ").split()))

start = int(input("Enter starting block: "))
length = int(input("Enter number of blocks required: "))

allocation = []

for i in range(length):

    block = start + i

    if block in blocks:

        allocation.append(block)

    else:

        print("File cannot be allocated")
        break

if len(allocation) == length:

    print("\nFile allocated successfully")

    print("Allocated Blocks:", allocation)

    print("File records are stored sequentially")

else:

    print("Insufficient consecutive blocks")
```

OUTPUT

```
= RESTART: C:/Users/ATHIN/AppData/Local/
sssssssssssssssssssssssssssssssssssssss
Enter available disk blocks: 4
Enter starting block: 3
Enter number of blocks required: 4
File cannot be allocated
Insufficient consecutive blocks
```

```python
n = int(input("Enter number of file blocks: "))

blocks = list(map(int, input("Enter file block numbers: ").split()))

index_block = int(input("Enter index block number: "))

print("\nIndex Block:", index_block)
print("File Blocks:", blocks)

print("\nIndex Block Contents:")

for i in range(len(blocks)):
    print("Entry", i + 1, "-> Block", blocks[i])

print("\nFile allocated successfully using indexed allocation")
```

OUTPUT

```
Enter number of file blocks: 4
Enter file block numbers: 34
Enter index block number: 5

Index Block: 5
File Blocks: [34]

Index Block Contents:
Entry 1 -> Block 34

File allocated successfully using indexed allocation
```

```
n = int(input("Enter number of file blocks: "))

blocks = list(map(int, input("Enter block numbers: ").split()))

print("\nLinked Allocation")

for i in range(n):
    if i == n - 1:
        print("Block", blocks[i], "-> NULL")
    else:
        print("Block", blocks[i], "->", blocks[i + 1])

print("\nFile allocated successfully using linked allocation")
```

OUTPUT

```
= RESTART: C:/Users/ATHIN/AppData/Local/Programs/Pyt
sssssssssssssssssssssssssssssssssssssssssssssssssss
Enter number of file blocks: 4
Enter block numbers: 3

Linked Allocation
```

```
requests = list(map(int, input("Enter disk requests: ").split()))

head = int(input("Enter initial head position: "))

total_movement = 0
current = head

print("\nDisk Head Movement:")

for request in requests:

    movement = abs(current - request)

    print(current, "->", request,
          "Movement =", movement)

    total_movement += movement

    current = request

print("\nTotal Head Movement:", total_movement)

print("Average Head Movement:",
      total_movement / len(requests))
```

OUTPUT

```
= RESTART: C:/Users/ATHIN/AppData/Local/Programs/Python/Python31
sssssssssssssssssssssssssssssssssssssssssssssssssssssssssss.py
Enter disk requests: 5
Enter initial head position: 4

Disk Head Movement:
4 -> 5 Movement = 1

Total Head Movement: 1
Average Head Movement: 1.0
```

```
left.sort()
right.sort()

sequence = []
current = head
movement = 0

if direction == "right":

    for request in right:
        movement += abs(current - request)
        sequence.append(request)
        current = request

    movement += abs(current - (disk_size - 1))
    current = disk_size - 1

    for request in reversed(left):
        movement += abs(current - request)
        sequence.append(request)
        current = request

else:

    for request in reversed(left):
        movement += abs(current - request)
        sequence.append(request)
        current = request

    movement += abs(current - 0)
    current = 0

    for request in right:
        movement += abs(current - request)
        sequence.append(request)
        current = request

print("\nService Sequence:")
print(sequence)

print("Total Head Movement:", movement)
```

OUTPUT

```
= RESTART: C:/Users/ATHIN/AppData/Local/Programs/Python/Python
sssssssssssssssssssssssssssssssssssssssssssssssssssssss.py
Enter disk requests: 4
Enter initial head position: 5
Enter disk size: 3
Enter direction (left/right): right

Service Sequence:
[4]
Total Head Movement: 5
```

```
    if request < head:
        left.append(request)
    else:
        right.append(request)

left.sort()
right.sort()

sequence = []
current = head
movement = 0

for request in right:

    movement += abs(current - request)

    sequence.append(request)

    current = request

movement += abs(current - (disk_size - 1))

current = disk_size - 1

movement += disk_size - 1

current = 0

for request in left:

    movement += abs(current - request)

    sequence.append(request)

    current = request

print("\nService Sequence:")
print(sequence)

print("Total Head Movement:", movement)
```

OUTPUT

```
= RESTART: C:/Users/ATHIN/AppData/Lo
sssssssssssssssssssssssssssssssssss
Enter disk requests: 5
Enter initial head position: 4
Enter disk size: 5

Service Sequence:
[5]
Total Head Movement: 6
```

```
print("Linux File Access Permissions")

print("\nUser Types:")
print("1. Owner")
print("2. Group")
print("3. Others")

print("\nFile Permissions:")
print("r = Read")
print("w = Write")
print("x = Execute")

print("\nPermission Values:")
print("Read    = 4")
print("Write   = 2")
print("Execute = 1")

print("\nExample:")
print("rwxr-xr--")

print("\nOwner  : rwx")
print("Group  : r-x")
print("Others : r--")

print("\nNumeric Representation:")
print("Owner  = 7")
print("Group  = 5")
print("Others = 4")

print("\nPermission = 754")
```

OUTPUT

```
= RESTART: C:/Users/ATHIN/AppData/Local/Progra
sssssssssssssssssssssssssssssssssssssssssssss
Linux File Access Permissions

User Types:
1. Owner
2. Group
3. Others

File Permissions:
r = Read
w = Write
x = Execute

Permission Values:
Read    = 4
Write   = 2
Execute = 1

Example:
rwxr-xr--

Owner  : rwx
Group  : r-x
Others : r--

Numeric Representation:
Owner  = 7
Group  = 5
Others = 4

Permission = 754
```